

Ampliación a 3D de un simulador peatonal: multiplanta y escaleras

Simulación de Sistemas — ITBA

Julio de 2026

Escenario de estudio: **Escuela** (2 plantas, aulas, pasillos, escaleras, recreo con kiosco).
Modelo físico: **Contractile Particle Model (CPM)** — *Anisotropic CPM with Avoidance*,
Baglietto–Parisi / Martin–Parisi.

1. Objetivo

El simulador de partida modelaba peatones en **2D**: la posición y la velocidad eran un tipo **Vec2** (x, y) usado en cascada por todos los módulos (física, geometría, grafo de navegación, detección de vecinos). El objetivo del TP fue **ampliarlo a 3D**: soportar mapas con **más de una planta** conectadas por **escaleras**, y estudiar dos sub-escenarios sobre un mapa de escuela — **Evacuación e Ingreso** — variando un input y midiendo un observable.

El hilo conductor de la ampliación es la coordenada z (la *planta*, y la altura intermedia mientras se recorre una escalera) y que **vecindad, física, grafo y salida** la respeten. Las escaleras son el elemento nuevo que conecta plantas.

2. El modelo 3D (resumen técnico)

La ampliación se resolvió con un enfoque **híbrido**: se introdujo un **Vec3** (x, y, z) para las posiciones/velocidades, pero se **conservó Vec2** como tipo planar de toda la dinámica horizontal (la física peatonal ocurre en el plano de cada planta; la z cambia sólo en la escalera). Las decisiones de diseño están registradas en `DECISIONES.md` (D1–D21); acá se resumen las centrales.

Módulo	Qué cambió en 3D	Dec.
Tipos base	Vec3 para posición/velocidad; <code>AgentState.z</code> . La z no es grado de libertad dinámico: se interpola por el avance sobre la escalera (no hay v_z).	D1, D2
Geometría	Cada pared/salida/aula/generador/server lleva su planta z . Nuevo tipo Stairs en <code>STAIRS.csv</code> . Consultas por planta.	D3–D5
Grafo de navegación	Una malla por planta unida por aristas de escalera (pie $\leftrightarrow$ tope). A^* con heurística euclídea 3D ; visibilidad y <i>Furthest Visible Point</i> por planta.	D6
Vecinos (CIM)	Una grilla por planta + un índice “puente” por escalera: vecindad independiente por planta salvo sobre las escaleras. Paredes y vértices se detectan como vecinos.	D8
Física (CPM)	Dinámica planar en la planta del agente. Sobre la escalera: velocidad reducida e interpolación de z ; anti-tunneling con las paredes de la planta actual.	D9
Salida	Se agrega la columna z : <code>tout; x; y; z; vx; vy; state; id</code> .	D10
Animación	Cada planta en 2D (círculos) + una vista 3D a 45° con las plantas apiladas.	—

Se mantuvo **CPM como único modelo** (se eliminó el SFM, D7). La suite de tests quedó en **149 verdes**.

3. La escalera realista (switchback con descanso)

La primera versión de la escalera era un **único tramo recto**. Se rediseñó para que fuera físicamente creíble (D19):

- **Switchback con descanso.** Cada escalera es un tramo en **L**: dos *flights* paralelos (A y B) unidos por un **piso de descanso** a media altura ($z = 1.5$ m). El agente sube el tramo A, gira en el descanso, y sube el tramo B hacia la planta siguiente. Se modeló como **dos Stairs encadenados a través del landing** (un “piso” chico a $z = 1.5$), reutilizando toda la maquinaria multiplanta sin reescribir el núcleo.
- **Confinamiento.** Barandas laterales en cada tramo y perímetro del landing (con huecos en las bocas) mantienen al agente dentro de la huella.
- **Carriles de subida/bajada (contraflujo).** Sobre tramos anchos el CPM aplica un *bias* lateral suave según la dirección de viaje, calibrado para no estrangular el flujo denso unidireccional.
- **Peldaños en la vista 3D.** La escalera se dibuja como un perfil de escalones (huella + alzada) en lugar de una línea inclinada.
- **Velocidad calibrada.** `speed_factor` = 0.38 sobre $v_d \approx 1.55$ m/s da una velocidad efectiva medida de ≈ 0.59 m/s en la escalera, con z monótona respecto del avance.

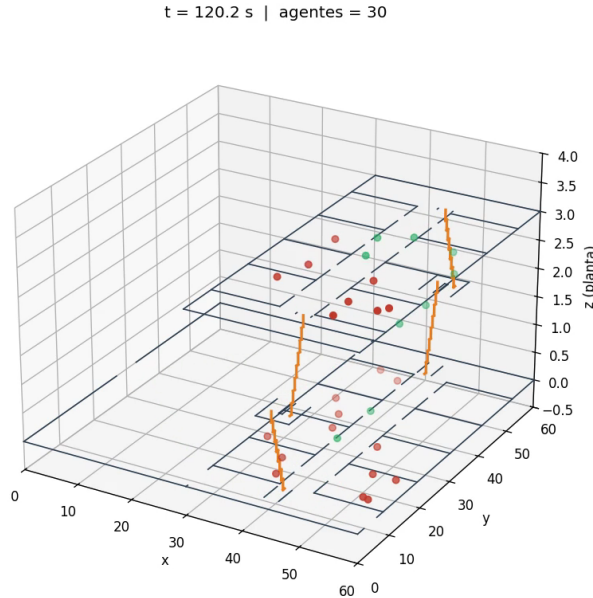


Figura 1: Vista 3D apilada de la Escuela ($t = 120$ s). Se ven las tres capas —PB ($z = 0$), descanso ($z = 1.5$) y P1 ($z = 3$)— y las dos escaleras *switchback* (peldaños en naranja), con agentes distribuidos en las tres.

Corrección de un artefacto (D21). Al inspeccionar la animación se detectó que algunos agentes que llegaban a la escalera **saltaban de $z = 0$ a $z \approx 0.56$ en un solo cuadro** (“se teletransportaban” a media escalera). El diagnóstico mostró que no era del render sino del ruteo: los agentes que se acercaban al pie por dentro de la huella recién enganchaban la z a mitad del tramo. Se resolvió (a) **excluyendo de la grilla del grafo los nodos dentro de la huella de las escaleras**, de modo que el agente entra por el pie con avance ≈ 0 , y (b) ajustando el punto en que el ruteo lo compromete a subir. El salto máximo por cuadro bajó de **0.56 m a 0.08 m** (imperceptible), sin regresar el ruteo multiplanta, verificado sobre el baseline y los sub-escenarios.

4. El escenario Escuela

Mapa cuadrado de 60×60 m con dos zonas:

- **Recreo** (izquierda, $x \in [0, 30]$, una sola planta $z = 0$): patio abierto con un **kiosco** (server con cola) y una salida propia.
- **Edificio** (derecha, $x \in [30, 60]$, dos plantas: **PB** $z = 0$ y **P1** $z = 3$): un pasillo vertical central separa **16 aulas** (4 por lado y por planta). **Dos escaleras switchback** en las puntas conectan PB $\leftrightarrow$ P1. La PB tiene salida propia y se comunica con el recreo por las esquinas; la P1 no tiene salida, se evacúa **bajando** por las escaleras.

Las **aulas son servers CLASSROOM** (recinto colectivo con una sesión de clase) y los planes se diferencian por planta para repartir la población sin sesgo (D15, D16).

5. Metodología: barridos reproducibles

El enunciado pide “scripts para ejecutar variando el input y generar los gráficos”. Se construyó una infraestructura de barrido reproducible:

- **Semilla** (`-Dsimped.seed`): se expuso una semilla determinista por stream (los módulos usaban `new Random()` sin sembrar, así que las réplicas no eran comparables).
- **tools/sweep_run.py**: compila una vez, genera un escenario por valor del input y corre la grilla (input $\times$ semillas), guardando cada `output.csv`.
- **tools/sweep_lib.py**: carga y agrega métricas desde el `output.csv` (trayectorias por agente, población por frame en una zona, tiempos de evacuación).
- **plot_evacuacion.py** y **plot_ingreso.py**: generan las figuras de los observables.

Todos los resultados son con **semilla 1** y modelo **CPM**.

6. Sub-escenario Evacuación

Planteo. Los alumnos arrancan **dentro de las aulas** (ambas plantas) y evacúan a las salidas; los de P1 bajan por las escaleras. **Input:** capacidad total N (barrido $N \in \{40, 80, 120\}$). **Observable primario:** distribución de tiempos de evacuación $t_{\text{evac}} = t_{\text{salida}} - t_{\text{inicio}}$ por agente. **Observable escalar:** t_{evac} promedio y máximo vs N .

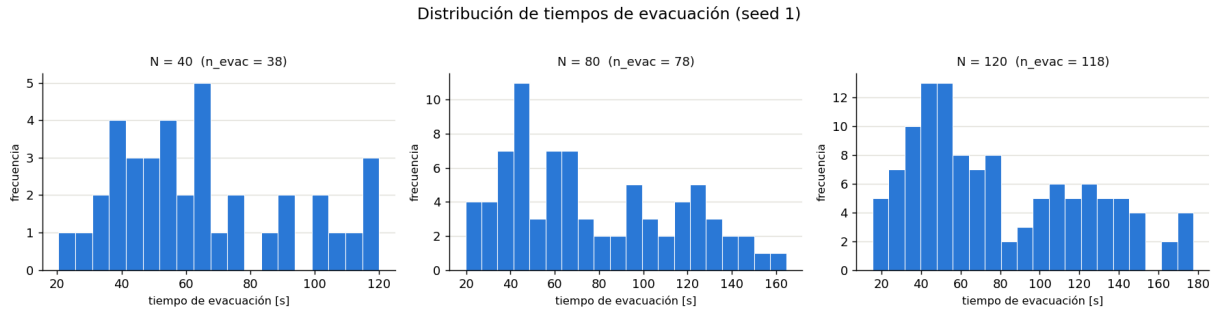


Figura 2: Distribución de tiempos de evacuación para $N = 40, 80$ y 120 .

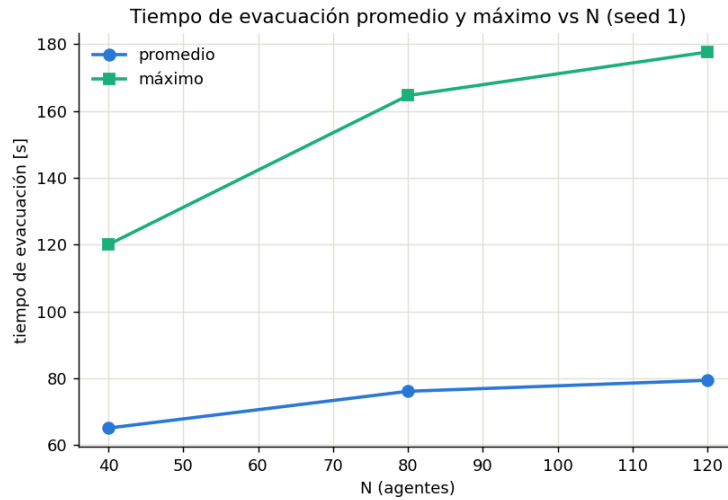


Figura 3: Tiempo de evacuación promedio y máximo en función de la capacidad N .

N	agentes evacuados	t_{evac} promedio [s]	t_{evac} máximo [s]
40	38	65.1	120.0
80	78	76.1	164.6
120	118	79.4	177.6

Lectura. El tiempo de evacuación crece con N (más agentes $\Rightarrow$ más congestión en pasillos y escaleras): el promedio pasa de ~ 65 s a ~ 79 s y el máximo de 120 s a 178 s al triplicar la capacidad. La distribución se ensancha y desarrolla una cola larga a mayor N (los últimos en salir, típicamente los de P1 que bajan la escalera, tardan cada vez más). Estos tiempos incorporan la escalera switchback (más larga y a velocidad reducida que un tramo recto).

7. Sub-escenario Ingreso

Planteo. Los $N_{\text{máx}} = 120$ alumnos **llegan** por las entradas y se dirigen a sus aulas; los que entran por el recreo pasan antes por el **kiosco**. **Input:** el **caudal**, es decir los 120 distribuidos en una ventana de llegada de **1, 5 o 10 minutos** ($N_{\text{máx}}$ fijo). **Observable primario:** población vs tiempo en una **zona de interés**. **Observable escalar:** ocupación máxima y promedio en esa zona vs caudal.

Elección de la zona. El enunciado sugiere “una zona, p.ej. antes de la escalera”. Se probó esa zona (el pie de la escalera principal) y **no muestra congestión**: la escalera switchback es ancha (dos carriles) y sólo la mitad de los alumnos suben a P1, repartidos entre las dos escaleras y a lo largo de toda la ventana, de modo que el pie queda casi vacío. La congestión real del Ingreso

se forma en la **cola del kiosco** (los ~ 60 alumnos del recreo se agolpan antes de clase), que da una señal fuerte y monótona. Se adoptó esa zona como observable (el “p.ej.” del enunciado lo habilita).

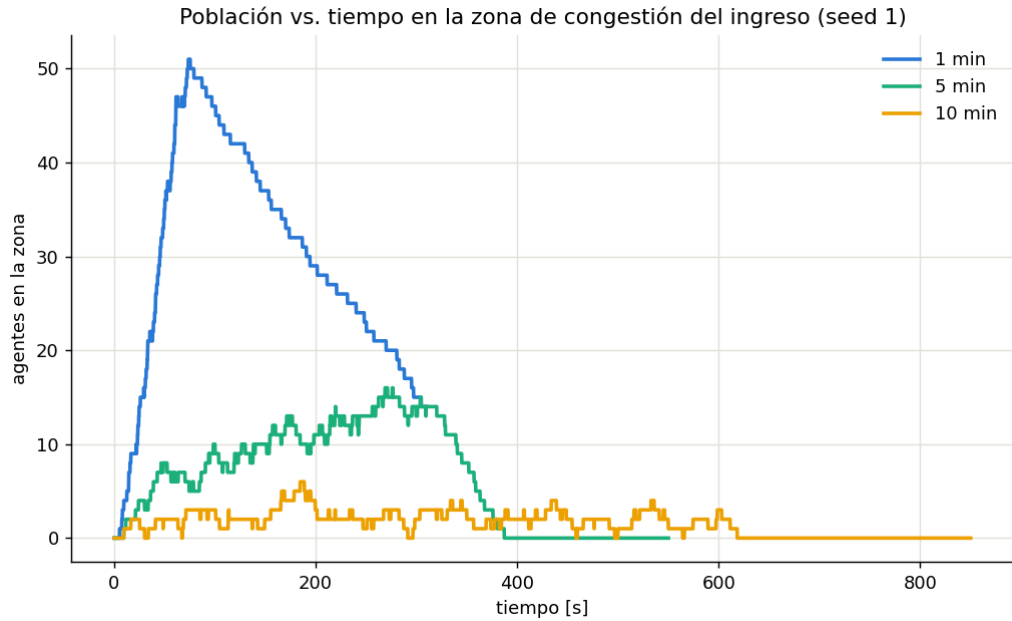


Figura 4: Población dentro de la zona de congestión (frente del kiosco) vs tiempo, para caudales de 1, 5 y 10 minutos.

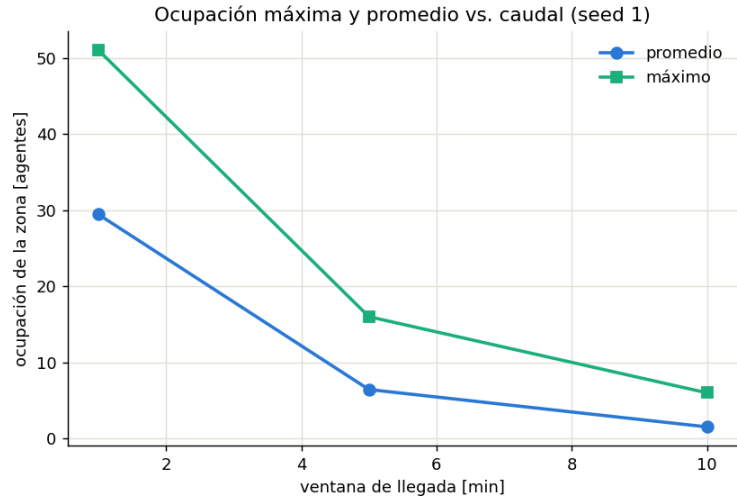


Figura 5: Ocupación máxima y promedio en la zona en función de la ventana de llegada (caudal).

Ventana de llegada	Caudal	Ocupación pico	Ocupación promedio
1 min	alto	51	29.4
5 min	medio	16	6.4
10 min	bajo	6	1.5

Lectura. Con $N_{\text{máx}}$ fijo, **cuanto más corto el caudal, mayor la congestión**: concentrar 120 alumnos en 1 minuto produce un pico de 51 agentes en la zona, mientras que repartirlos en 10 minutos lo baja a 6 (una reducción de $\sim 8.5\times$). La curva de población muestra el pico agudo

y temprano del caudal de 1 minuto frente a la meseta baja y extendida del de 10 minutos — el mismo total de personas, distinta intensidad de aglomeración.

7.1. Estudio complementario: variar la cantidad de agentes

Sobre el mismo sub-escenario se agregó un eje ortogonal — variar $N_{\text{máx}}$ a ventana fija (5 min) — con `tools/run_ingreso_nmax.py`, para ver cómo escala la congestión con la cantidad de gente.

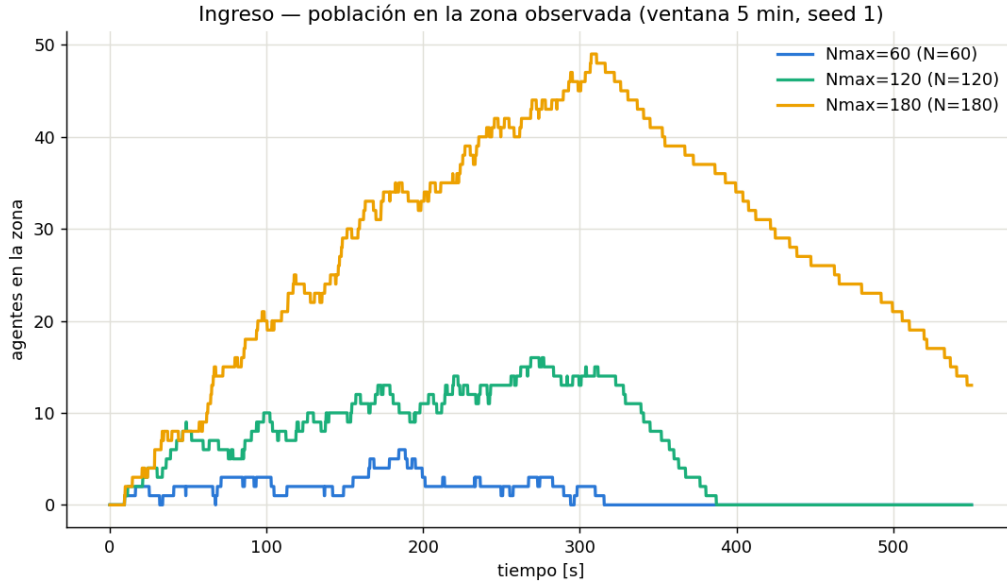


Figura 6: Población en la zona vs tiempo para $N_{\text{máx}} = 60, 120$ y 180 (ventana de 5 min).

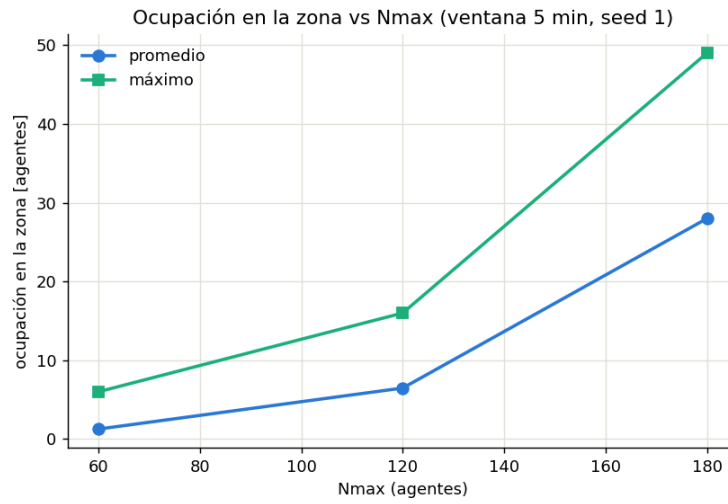


Figura 7: Ocupación máxima y promedio en la zona en función de $N_{\text{máx}}$.

$N_{\text{máx}}$	Ocupación pico	Ocupación promedio
60	6	1.3
120	16	6.5
180	49	28.0

La ocupación crece de forma marcadamente **supralineal** con $N_{\text{máx}}$ (el pico se multiplica por ~ 8 al triplicar la población): la zona no absorbe el flujo proporcionalmente, sino que se satura.

8. Mapas de calor (observable complementario)

Como extra opcional del enunciado se agregaron mapas de calor por planta (`tools/heatmaps.py`), con **colormaps secuenciales de un solo hue** (claro→oscuro) y las paredes superpuestas; las celdas nunca ocupadas quedan en gris.

Densidad de ocupación. Para cada celda de una grilla de 1×1 m se cuenta la ocupación media (agentes dentro de la celda promediados sobre todos los cuadros del **output**). En la **evacuación** (Figura 8) los puntos calientes son el **pasillo central** y los **pies de las escaleras** —por donde todos convergen para bajar— más las dos rutas hacia las salidas. En el **ingreso** (Figura 9) el punto caliente es la **cola del kiosco** (el pico de congestión que ya mostraba el observable del sub-escenario), con los pasillos y entradas mucho más tenues.

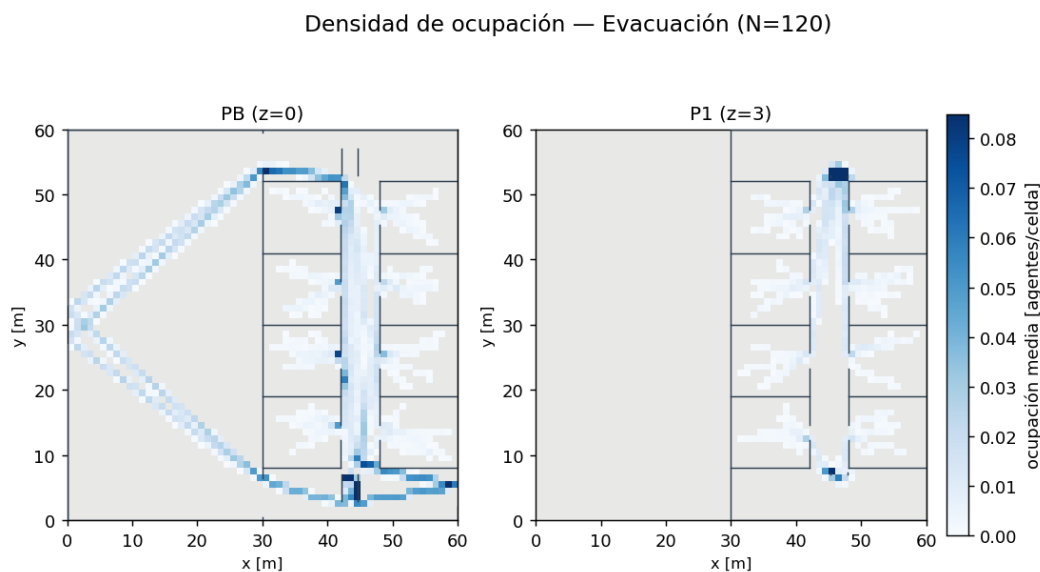


Figura 8: Densidad de ocupación durante la evacuación ($N = 120$): el pasillo central y las escaleras concentran el flujo.

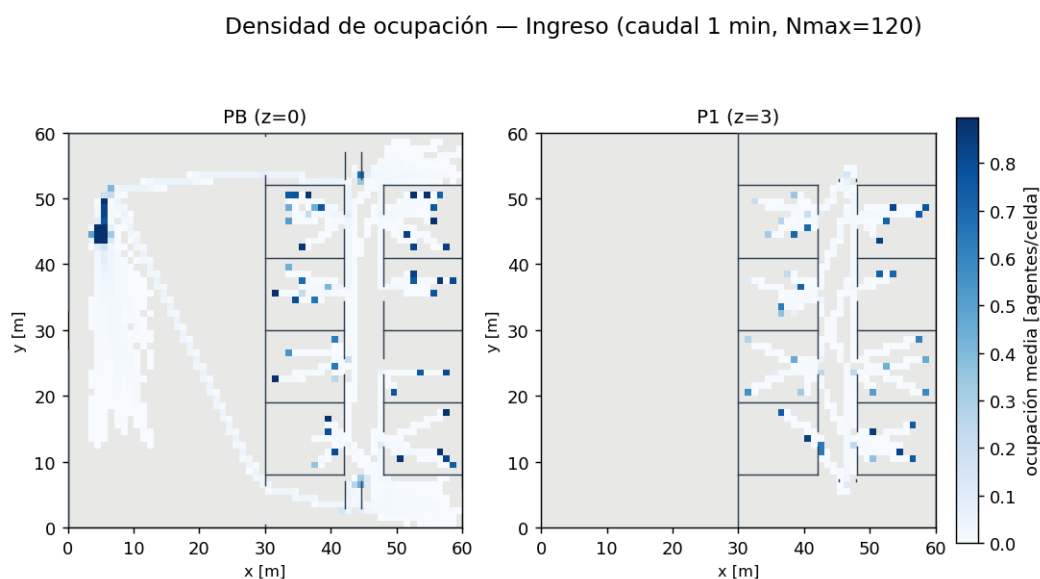


Figura 9: Densidad de ocupación durante el ingreso (caudal de 1 min): la cola del kiosco es el punto caliente.

Tiempos de evacuación por origen. Se toma el punto de **origen** de cada agente (su primer cuadro) y se colorea la celda con el **tiempo de evacuación medio** de los agentes que arrancan ahí (Figura 10). El resultado es nítido: las aulas de **PB** evacúan en $\sim 40\text{--}60$ s (naranja claro), mientras que las de **P1** tardan $\sim 120\text{--}170$ s (rojo oscuro), porque deben **bajar la escalera**. El mapa cuantifica espacialmente el costo de estar en la planta alta.

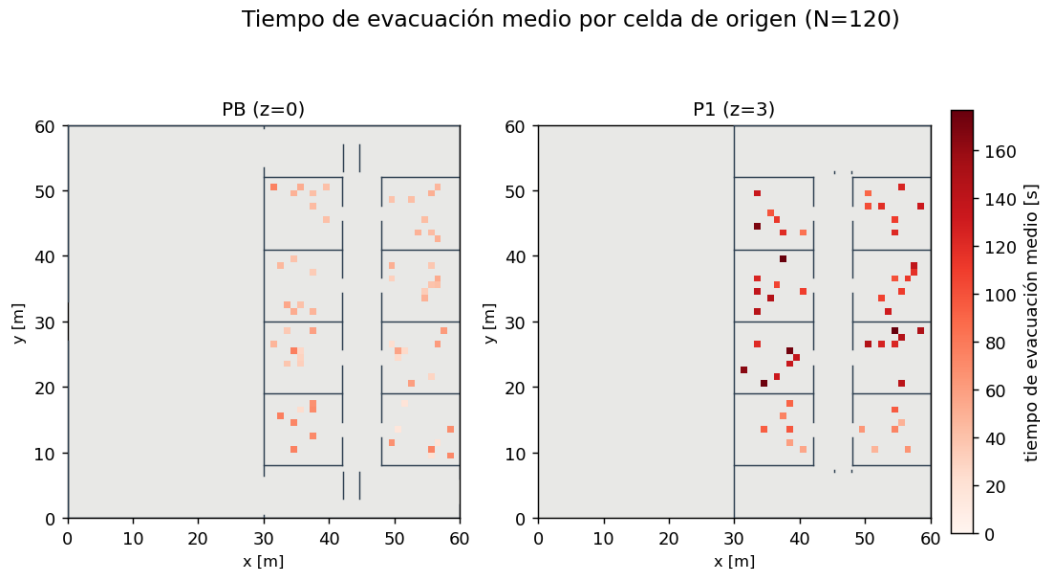


Figura 10: Tiempo de evacuación medio según la celda de origen ($N = 120$). Las aulas de P1 (rojo oscuro) evacúan mucho más lento que las de PB por tener que descender por las escaleras.

9. Verificación y calidad

- **Suite de tests: 149 verdes** (8 *skipped*), incluidos los de la física de escaleras (CpmOperationalModelStair) del grafo multiplanta (GraphMultiFloorTest) y del CIM por planta.
- **Baseline “día escolar”** (ingreso $\rightarrow$ clase $\rightarrow$ evacuación) validado: de ~ 30 alumnos, ~ 15 suben a P1 por las escaleras y bajan, **0 quedan atascados** en el descanso, y $\sim 28/30$ evacúan.
- Cada resultado fue **medido directamente sobre el output.csv** de una corrida con semilla fija, no asumido.

10. Conclusiones

- El simulador quedó **ampliado a 3D** de punta a punta: agentes con $z \neq 0$ que recorren escaleras a velocidad reducida, detección de vecinos y física por planta, grafo de navegación 3D (A* euclídeo con unión por escaleras), salida con z y animación 2D-por-planta + 3D apilada.
- La **escalera switchback** con descanso, confinamiento, carriles de contraflujo y peldaños da un comportamiento físicamente creíble; el artefacto del salto de z se diagnosticó y resolvió.
- Los dos sub-escenarios producen observables con **sentido físico claro**: la evacuación se alarga con la capacidad, y la congestión de ingreso crece al concentrar el caudal (y al aumentar $N_{\text{máx}}$).
- Todas las decisiones de arquitectura quedaron registradas en DECISIONES.md (D1–D21).

A. Reproducción de los resultados

```
# Compilar y tests
mvn clean test                                # 149 verdes

# Escenario Escuela (baseline) + animacion 3D
python tools/scenarios-builders/build_escuela.py
mvn -q dependency:build-classpath -Dmdep.outputFile=out/.cp.txt
java -Dsimped.seed=1 -cp "target/classes:$(cat out/.cp.txt)" \
    ar.edu.itba.simped.App scenarios/escuela out/escuela.csv cpm
python tools/visualize_simulation_3d.py --scenario scenarios/escuela \
    --output out/escuela.csv --out out/escuela_3d.mp4 --stride 3

# Evacuacion: barrido N en {40, 80, 120} + figuras
python tools/sweep_run.py --mode evacuacion --values 40,80,120 --seeds 1 --om cpm
python tools/plot_evacuacion.py --sweep-dir out/sweeps/evacuacion --seed 1 --out-prefix out/
    evac

# Ingreso: barrido caudal 1/5/10 min + figuras
python tools/sweep_run.py --mode ingreso --values 1,5,10 --seeds 1 --om cpm
python tools/plot_ingreso.py --sweep-dir out/sweeps/ingreso --seed 1 --out-prefix out/ingreso

# Ingreso variando la cantidad de agentes (estudio complementario)
python tools/run_ingreso_nmax.py --nmax 60,120,180 --window 5 --seed 1
```